

# Database ERD Analysis & Entity Mapping

This database Entity-Relationship (ER) diagram represents a highly mature, double-entry adjacent ledger design. It is perfectly optimized to handle asynchronous bill splitting, shared household expenses, and direct interpersonal debts.

Below is an in-depth breakdown of what each entity represents, how they map to one another (including their cardinalities), and a technical evaluation of why this design succeeds.

## 1. Entity Definitions & Technical Meanings

### ACCOUNTS

- **What it represents:** The physical or virtual financial repositories where money actually resides or moves.
- **Examples:** Your credit card, checking account, cash pocket, or savings account.
- **Role in your system:** It records the source of cash flow for your transactions.

### IMPORTS

- **What it represents:** Metadata tracking for batch file ingestion sessions.
- **Examples:** A record of a credit card CSV statement uploaded on a specific Sunday.
- **Role in your system:** Allows you to manage bulk data entries. If a CSV upload goes wrong, you can easily identify, quarantine, or delete all transactions linked to that specific import ID.

### TRANSACTIONS

- **What it represents:** Real-world payment events (cash flow occurrences).
- **Examples:** A \$120 charge at Costco or a \$30 pizza purchase.
- **Role in your system:** This is your unprocessed "holding area" or "inbox." It records that money moved, who paid it, and how it was paid, but does not dictate who consumed it yet.

### ALLOCATIONS

- **What it represents:** Consumption ownership. This is the heart of your system.
- **Examples:** Splitting a \$120 Costco transaction into three \$40 consumption lines.
- **Role in your system:** This table breaks a single transaction into individual shares. Your personal \$800 monthly budget is calculated by summing only your personal allocations.

### CATEGORIES

- **What it represents:** Classifications for budget reporting.
- **Examples:** "Groceries", "Transit", "Eating Out", or "Utilities".



- **Role in your system:** Categorizes individual allocations (e.g., your \$40 share of Costco goes to "Groceries", while roommate shares are categorized as external IOUs).

## PEOPLE

- **What it represents:** The individuals in your shared financial ecosystem.
- **Examples:** You (Darshan), Logan, Rahul, or Ankit.
- **Role in your system:** Used to assign who paid for transactions, who consumed allocations, and who is sending or receiving settlements.

## SETTLEMENTS

- **What it represents:** Debt-clearing events that occur outside of standard consumer transactions.
- **Examples:** Logan sending you a \$20 e-transfer or handing you a \$10 bill.
- **Role in your system:** Allows you to reconcile balances directly. It tracks money moving from person to person to settle outstanding IOUs without creating fake purchase transactions.

## GROUPS & GROUP\_MEMBERS

- **What it represents:** A classic many-to-many relationship structure.
- **Examples:** A group called "The House" containing you, Logan, and Rahul.
- **Role in your system:** Grouping people makes splitting easier. Instead of manually adding three people to a grocery split every time, you can assign an allocation to a group, and the system can look up the members automatically.

## 2. Relationship & Cardinality Mapping

The ER diagram utilizes standard Crow's Foot notation. Here is how the lines connect and what they mean mathematically.

Notation Legend:

```

|| = Exactly One (Mandatory One)
o| = Zero or One (Optional One)
o< = Zero or Many (Optional Many)

```

### 1. ACCOUNTS to TRANSACTIONS

- **Relationship:** `paid via (optional)`
- **Cardinality:** `ACCOUNTS o| ----- o< TRANSACTIONS`
- **Meaning:** \* A transaction is optionally paid via an account (zero or one). This is a vital design choice. It allows you to log manual transactions (like "Rahul paid for my transit") without forcing the transaction to link to your credit card.
  - An account can be used to pay for zero, one, or many transactions over time.

## 2. IMPORTS to TRANSACTIONS

- **Relationship:** `creates`
- **Cardinality:** IMPORTS || ----- o< TRANSACTIONS
- **Meaning:**
  - Every imported transaction belongs to exactly one import session.
  - An import session can create zero, one, or many transactions in your inbox.

## 3. PEOPLE to TRANSACTIONS

- **Relationship:** `pays (payer_person_id)`
- **Cardinality:** PEOPLE || ----- o< TRANSACTIONS
- **Meaning:**
  - Every transaction must have exactly one person designated as the payer (the person who swiped their card or handed over physical cash).
  - A person can be the payer for zero, one, or many transactions.

## 4. TRANSACTIONS to ALLOCATIONS

- **Relationship:** `is split into`
- **Cardinality:** TRANSACTIONS || ----- o< ALLOCATIONS
- **Meaning:**
  - Each individual allocation must belong to exactly one master transaction.
  - A master transaction is split into zero, one, or many allocations (unsplit transactions have zero allocations, while a split transaction has multiple).

## 5. CATEGORIES to ALLOCATIONS

- **Relationship:** `classifies`
- **Cardinality:** CATEGORIES || ----- o< ALLOCATIONS
- **Meaning:**
  - Each allocation must belong to exactly one category.
  - A category can classify zero, one, or many allocations.

## 6. PEOPLE to ALLOCATIONS

- **Relationship:** `consumes`
- **Cardinality:** PEOPLE || ----- o< ALLOCATIONS
- **Meaning:**
  - Each allocation must be consumed by exactly one person.
  - A person can consume zero, one, or many allocations.

## 7. PEOPLE to SETTLEMENTS (Double Relationship)

- **Relationship:** `from_person` and `to_person`
- **Cardinality:** PEOPLE || ----- o< SETTLEMENTS
- **Meaning:**
  - Every settlement record must have exactly one sender ( `from_person` ) and exactly one receiver ( `to_person` ).
  - A person can be the sender or receiver of zero, one, or many settlements.

## 8. GROUPS & PEOPLE (via GROUP\_MEMBERS)

- **Relationship:** Many-to-Many association.
- **Cardinality:** \* GROUPS || ----- o< GROUP\_MEMBERS
  - PEOPLE || ----- o< GROUP\_MEMBERS
- **Meaning:**
  - A group can have zero, one, or many members (represented by records in the junction table `GROUP_MEMBERS` ).
  - A person can belong to zero, one, or many groups.
  - Each record in `GROUP_MEMBERS` connects exactly one person to exactly one group.

## 3. Why This Database Model Succeeds

This design is highly resilient because it solves three main flaws that plague typical consumer finance applications:

1. **It supports the "Someone Else Paid" scenario natively:** If Logan pays \$30 for pizza and your share is \$15, you can create a Transaction where `payer_person_id` is Logan, and the payment account is null. You then split it into two allocations (\$15 for you, \$15 for Logan). Your local system instantly knows you spent \$15 (which hits your \$800 limit) and that you owe Logan \$15.
2. **It simplifies the \$800 budget math:** To calculate your monthly progress, you do not look at transactions or bank statements. You simply run a query summing the `amount` from the `allocations` table where `person_id` is you, `category` is not an IOU category, and the transaction date falls within the current billing cycle.
3. **It keeps a clean Audit Trail:** By separating transactions from settlements, you never have to "fake" grocery purchases to account for e-transfers or cash handovers. A cash handover is logged as a clean settlement, decreasing outstanding debt without touching your budget limit.